

Иерархическая кластеризация (AgglomerativeClustering)

Когда использовать: В биологии для построения деревьев видов (дендрограмм), классификации документов по темам, когда нужна визуализация вложенности кластеров.

- **Параметры:** `n_clusters` или `distance_threshold`, `linkage` (тип связи: `ward`, `complete`, `average`).
- **Геометрия:** Может использовать любые попарные расстояния.
- **Особенность:** Трансдуктивный, отлично визуализируется дендрограммой.

Пример: Кластеризация стран по признакам

Задача: Сгруппировать страны по уровню ВВП и грамотности, чтобы увидеть иерархию групп.

```
import numpy as np

import matplotlib.pyplot as plt

from sklearn.cluster import AgglomerativeClustering
from scipy.cluster.hierarchy import dendrogram, linkage

from sklearn.datasets import load_iris # Для примера возьмем простые данные
# 1. Данные (Iris dataset как пример)

data = load_iris()

X = data.data[:10] # Возьмем 10 образцов для простоты дендрограммы

# 2. Построение дендрограммы (чтобы визуально выбрать кол-во кластеров)
# Используем linkage из scipy

linked = linkage(X, method='ward')

plt.figure(figsize=(10, 7))

dendrogram(linked,

            orientation='top',

            labels=data.target[:10],

            distance_sort='descending',

            show_leaf_counts=True)

plt.title('Дендрограмма')
```

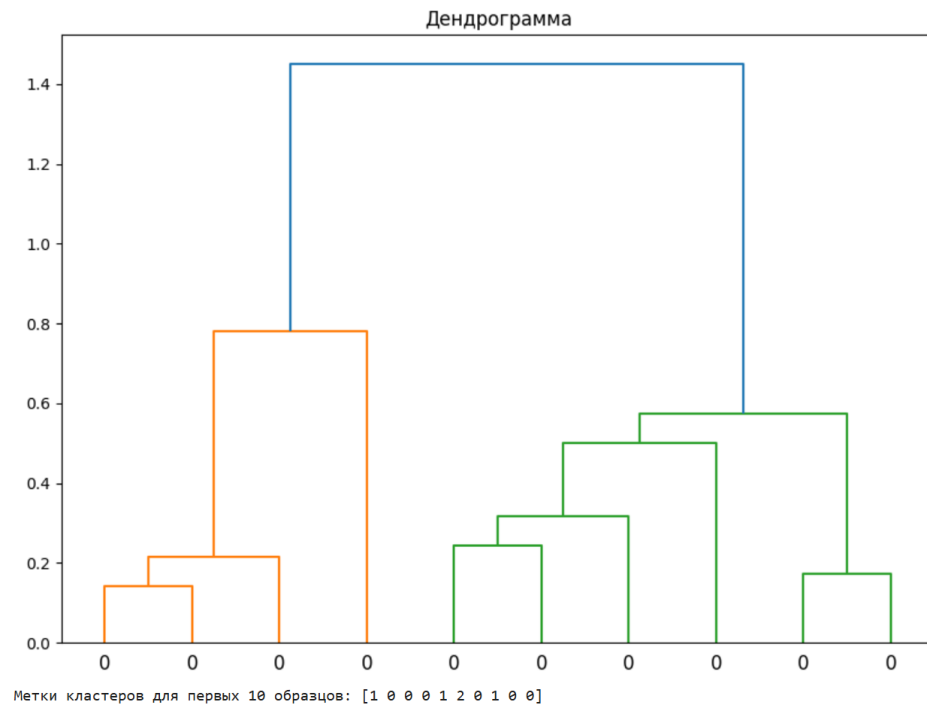
```
plt.show()
```

```
# 3. Запуск кластеризации (например, разбиваем на 3 кластера)
```

```
model = AgglomerativeClustering(n_clusters=3, linkage='ward')
```

```
labels = model.fit_predict(X)
```

```
print("Метки кластеров для первых 10 образцов:", labels)
```



Проверка модели: Визуальный осмотр дендрограммы: Выбирается уровень (расстояние) на котором "срезается" дерево, чтобы получить нужное количество кластеров. **Coherence Score (для текстов):** если кластеризуется текст, можно оценить, насколько слова внутри одного кластера семантически близки.

Суть: Строит дерево вложенных кластеров.

```
from sklearn.cluster import AgglomerativeClustering
```

```
from sklearn.datasets import make_blobs
```

```
import matplotlib.pyplot as plt # Данные
```

```
X, _ = make_blobs(n_samples=300, centers=4, random_state=0) # Модель
```

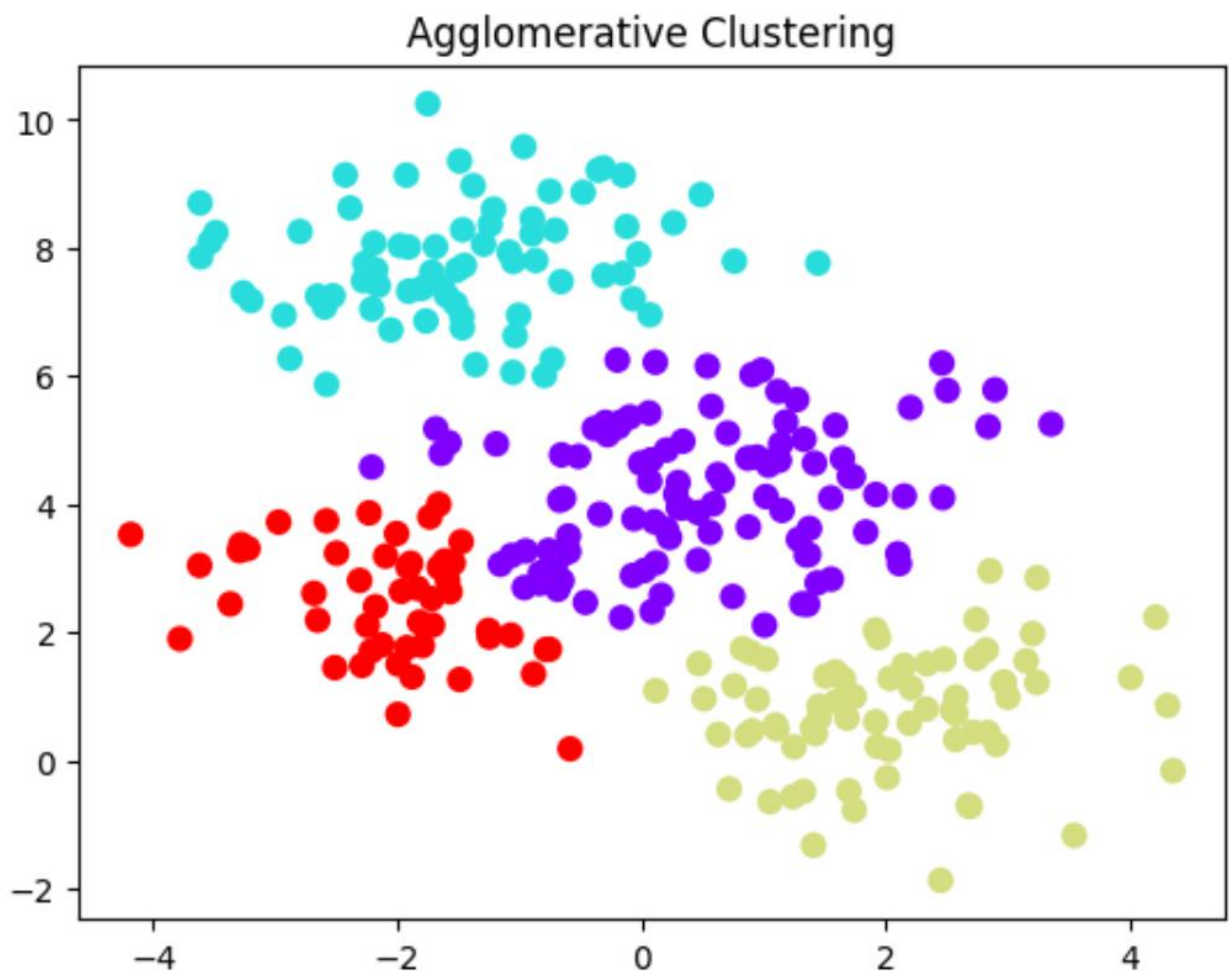
```
agg = AgglomerativeClustering(n_clusters=4)
```

```
labels = agg.fit_predict(X) # Визуализация
```

```
plt.scatter(X[:, 0], X[:, 1], c=labels, s=50, cmap='rainbow')
```

```
plt.title("Agglomerative Clustering")
```

```
plt.show()
```



Иерархическая кластеризация в Python

1. Основные методы иерархической кластеризации

Методы связи (Linkage Methods):

- **Single linkage** - минимальное расстояние между ближайшими точками кластеров
- **Complete linkage** - максимальное расстояние между точками кластеров
- **Average linkage** - среднее расстояние между всеми точками кластеров
- **Ward linkage** - минимизация внутрикластерной дисперсии

2. Пример программы на Python

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import AgglomerativeClustering
from sklearn.datasets import make_blobs, make_moons
from scipy.cluster.hierarchy import dendrogram, linkage
```

```

from sklearn.neighbors import kneighbors_graph
import pandas as pd

# Пример 1: Базовый пример с синтетическими данными
print("=== ПРИМЕР 1: Базовый пример ===")

# Генерация данных
X, y = make_blobs(n_samples=100, centers=3, cluster_std=0.8, random_state=42)

# Создание и обучение модели с разными методами связи
methods = ['ward', 'complete', 'average', 'single']

fig, axes = plt.subplots(2, 2, figsize=(12, 10))

for idx, method in enumerate(methods):

    # Иерархическая кластеризация
    clustering = AgglomerativeClustering(n_clusters=3, linkage=method)
    labels = clustering.fit_predict(X)

    # Визуализация
    ax = axes[idx // 2, idx % 2]

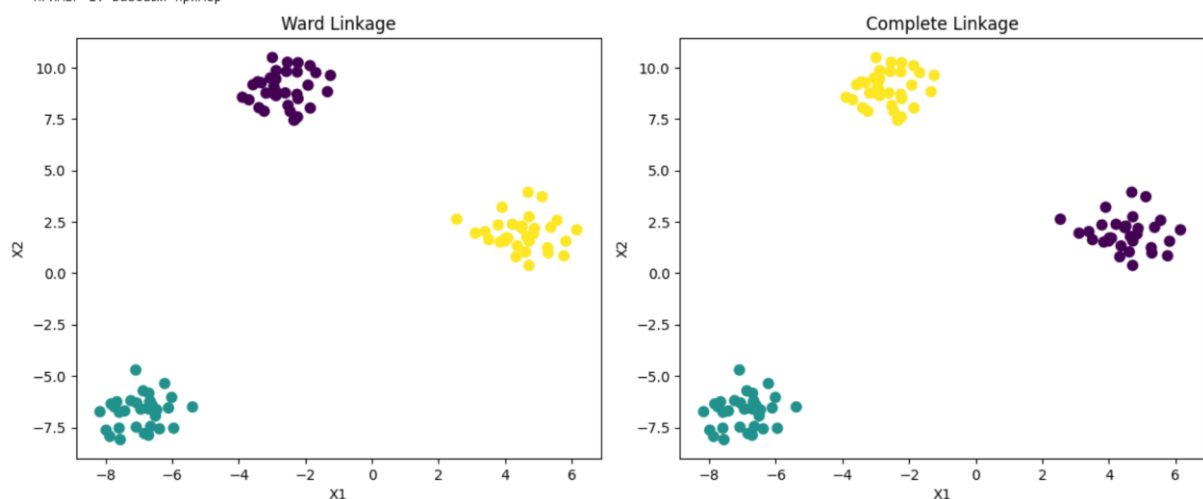
    scatter = ax.scatter(X[:, 0], X[:, 1], c=labels, cmap='viridis', s=50)

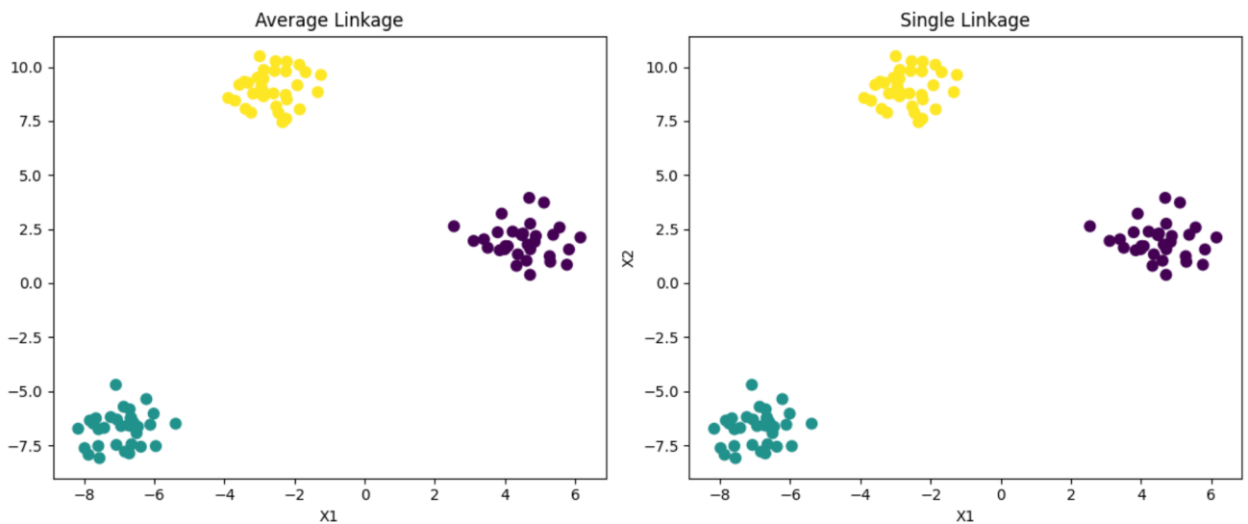
    ax.set_title(f'{method.capitalize()} Linkage')
    ax.set_xlabel('X1')
    ax.set_ylabel('X2')

plt.tight_layout()
plt.show()

```

=== ПРИМЕР 1: Базовый пример ===





3. Дендрограмма (визуализация иерархии)

Пример 2: Дендрограмма для визуализации иерархии

```
print("\n=== ПРИМЕР 2: Дендрограмма ===")
```

Вычисление матрицы связей

```
linkage_matrix = linkage(X, method='ward')
```

Построение дендрограммы

```
plt.figure(figsize=(12, 6))
```

```
dendrogram(linkage_matrix, truncate_mode='level', p=5)
```

```
plt.title('Дендрограмма иерархической кластеризации (Ward)')
```

```
plt.xlabel('Индексы образцов')
```

```
plt.ylabel('Расстояние')
```

```
plt.show()
```

=== ПРИМЕР 2: Дендрограмма ===



4. Кластеризация с ограничениями связности

Пример 3: Кластеризация с ограничениями связности

```
print("\n=== ПРИМЕР 3: С ограничениями связности ===")
```

Генерация данных в форме полумесяцев

```
X_moons, y_moons = make_moons(n_samples=200, noise=0.05,  
random_state=42)
```

Создание графа связности (только 10 ближайших соседей)

```
connectivity = kneighbors_graph(X_moons, n_neighbors=10, include_self=False)
```

Кластеризация без ограничений и с ограничениями

```
fig, axes = plt.subplots(1, 2, figsize=(12, 5))
```

Без ограничений

```
clustering_no_conn = AgglomerativeClustering(n_clusters=2, linkage='single')
```

```
labels_no_conn = clustering_no_conn.fit_predict(X_moons)
```

```
axes[0].scatter(X_moons[:, 0], X_moons[:, 1], c=labels_no_conn, cmap='viridis',  
s=50)
```

```
axes[0].set_title('Single Linkage без ограничений')
```

С ограничениями

```
clustering_conn = AgglomerativeClustering(n_clusters=2, linkage='single',  
connectivity=connectivity)
```

```
labels_conn = clustering_conn.fit_predict(X_moons)
```

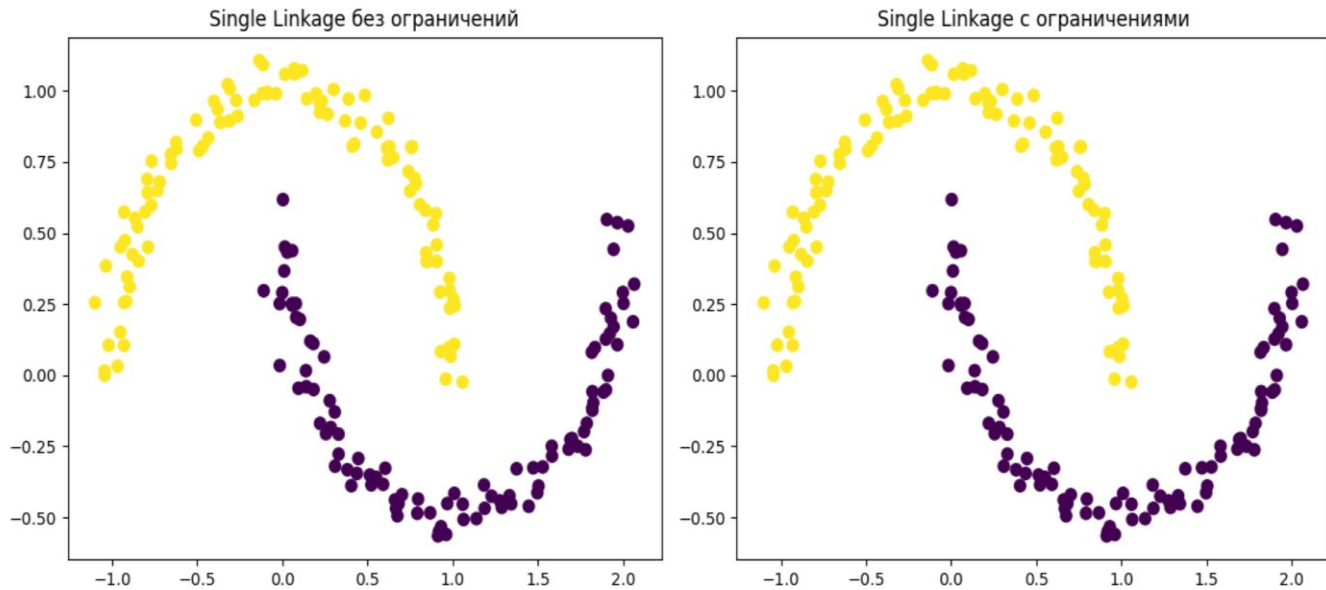
```
axes[1].scatter(X_moons[:, 0], X_moons[:, 1], c=labels_conn, cmap='viridis', s=50)
```

```
axes[1].set_title('Single Linkage с ограничениями')
```

```
plt.tight_layout()
```

```
plt.show()
```

=== ПРИМЕР 3: С ограничениями связности ===



5. Feature Agglomeration (агломерация признаков)

Пример 4: Feature Agglomeration - уменьшение размерности

```
print("\n=== ПРИМЕР 4: Агломерация признаков ===")
```

```
from sklearn.cluster import FeatureAgglomeration
```

```
from sklearn.datasets import load_iris
```

Загрузка данных Iris

```
iris = load_iris()
```

```
X_iris = iris.data
```

```
feature_names = iris.feature_names
```

```
print(f'Исходное количество признаков: {X_iris.shape[1]}')
```

Агломерация признаков

```
agglo = FeatureAgglomeration(n_clusters=2)
```

```
X_reduced = agglo.fit_transform(X_iris)
```

```
print(f'Количество признаков после агломерации: {X_reduced.shape[1]}')
```

```
print(f'Группировка признаков: {agglo.labels_}')
```

```
for i, name in enumerate(feature_names):
```

```
    print(f'{name} -> группа {agglo.labels_[i]}')
```

==== ПРИМЕР 4: Агломерация признаков ====

Исходное количество признаков: 4

Количество признаков после агломерации: 2

Группировка признаков: [1 0 0 0]

sepal length (cm) -> группа 1

sepal width (cm) -> группа 0

petal length (cm) -> группа 0

petal width (cm) -> группа 0

6. Сравнение методов связи на разных типах данных

Пример 5: Сравнение методов связи на разных данных

```
print("\n=== ПРИМЕР 5: Сравнение методов на разных типах данных ===")
```

Генерация разных типов данных

```
datasets = [  
    ("Круговые кластеры", make_blobs(n_samples=150, centers=3,  
    random_state=42)[0]),  
    ("Полумесяцы", make_moons(n_samples=150, noise=0.05,  
    random_state=42)[0]),  
    ("Концентрические круги", np.random.randn(150, 2) * 2)  
]  
methods = ['ward', 'complete', 'average', 'single']  
results = []  
for data_name, X_data in datasets:  
    for method in methods:  
        try:  
            clustering = AgglomerativeClustering(n_clusters=2, linkage=method)  
            labels = clustering.fit_predict(X_data)  
            # Оценка качества (просто считаем уникальные метки)  
            n_clusters_found = len(np.unique(labels))  
            results.append({  
                'data': data_name,  
                'method': method,
```



```

        'n_clusters': n_clusters_found
    })
except:
    results.append({
        'data': data_name,
        'method': method,
        'n_clusters': 'error'
    })

```

Вывод результатов

```

df_results = pd.DataFrame(results)
print(df_results)

```

=== ПРИМЕР 5: Сравнение методов на разных типах данных ===

	data	method	n_clusters
0	Круговые кластеры	ward	2
1	Круговые кластеры	complete	2
2	Круговые кластеры	average	2
3	Круговые кластеры	single	2
4	Полумесяцы	ward	2
5	Полумесяцы	complete	2
6	Полумесяцы	average	2
7	Полумесяцы	single	2
8	Концентрические круги	ward	2
9	Концентрические круги	complete	2
10	Концентрические круги	average	2
11	Концентрические круги	single	2

7. Bisecting K-Means (разделяющий K-средних)

Пример 6: Bisecting K-Means (если установлен пакет)

```

print("\n=== ПРИМЕР 6: Bisecting K-Means ===")

```

try:

```
from sklearn.cluster import BisectingKMeans

# Генерация данных
X, y = make_blobs(n_samples=300, centers=4, cluster_std=0.8,
random_state=42)

# Сравнение стратегий разделения
fig, axes = plt.subplots(1, 2, figsize=(12, 5))

# Стратегия: самый большой кластер
bisect_largest = BisectingKMeans(n_clusters=4,
bisecting_strategy='largest_cluster', random_state=42)
labels_largest = bisect_largest.fit_predict(X)
axes[0].scatter(X[:, 0], X[:, 1], c=labels_largest, cmap='viridis', s=50)
axes[0].set_title('BisectingKMeans - largest_cluster')
axes[0].scatter(bisect_largest.cluster_centers_[:, 0],
bisect_largest.cluster_centers_[:, 1],
c='red', marker='X', s=200, label='Центры')

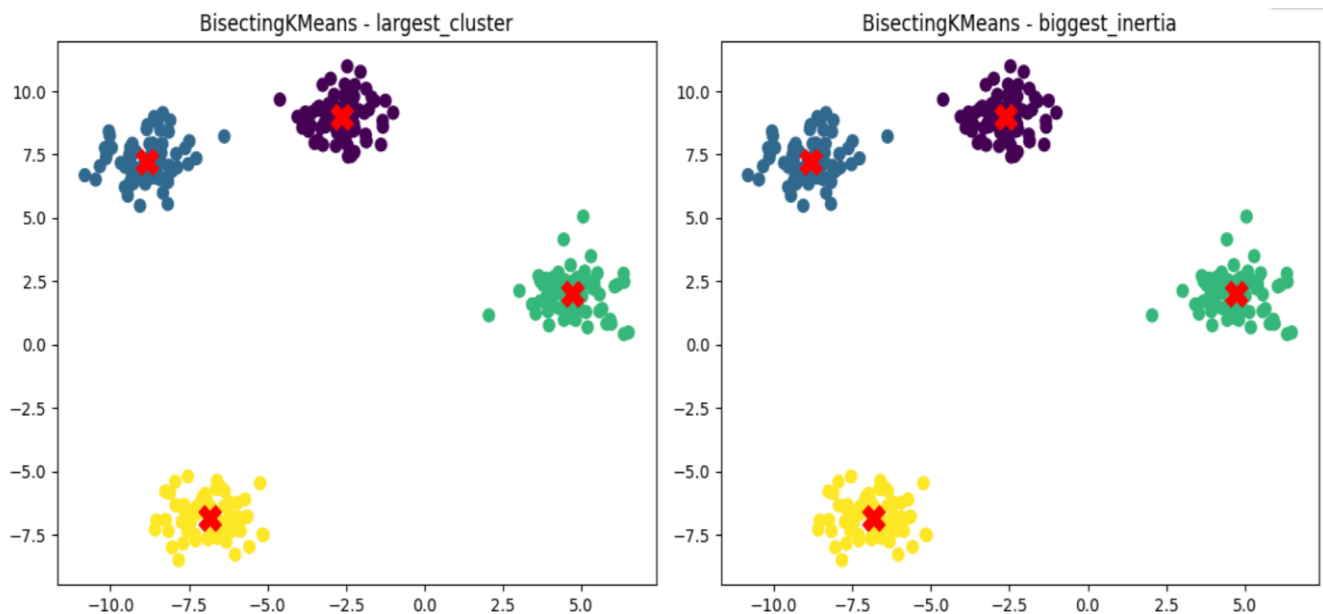
# Стратегия: наибольшая инерция
bisect_inertia = BisectingKMeans(n_clusters=4,
bisecting_strategy='biggest_inertia', random_state=42)
labels_inertia = bisect_inertia.fit_predict(X)
axes[1].scatter(X[:, 0], X[:, 1], c=labels_inertia, cmap='viridis', s=50)
axes[1].set_title('BisectingKMeans - biggest_inertia')
axes[1].scatter(bisect_inertia.cluster_centers_[:, 0],
bisect_inertia.cluster_centers_[:, 1],
c='red', marker='X', s=200, label='Центры')

plt.tight_layout()
plt.show()
```

except ImportError:

```
print("BisectingKMeans доступен в scikit-learn версии 1.1+")
```

=== ПРИМЕР 6: Bisecting K-Means ===



8. Задачи для проверки модели. Задача 1: Оптимальное число кластеров

```
def find_optimal_clusters(X, max_clusters=10):
```

```
    """
```

```
    Нахождение оптимального числа кластеров методом "локтя"
```

```
    """
```

```
    inertias = []
```

```
    for n in range(1, max_clusters + 1):
```

```
        clustering = AgglomerativeClustering(n_clusters=n, linkage='ward')
```

```
        labels = clustering.fit_predict(X)
```

```
        # Расчет инерции (сумма квадратов расстояний до центров)
```

```
        inertia = 0
```

```
        for i in range(n):
```

```
            cluster_points = X[labels == i]
```

```
            if len(cluster_points) > 0:
```

```
                center = cluster_points.mean(axis=0)
```

```
                inertia += np.sum((cluster_points - center) ** 2)
```

```
        inertias.append(inertia)
```

```
    # Визуализация
```

```
    plt.figure(figsize=(8, 4))
```

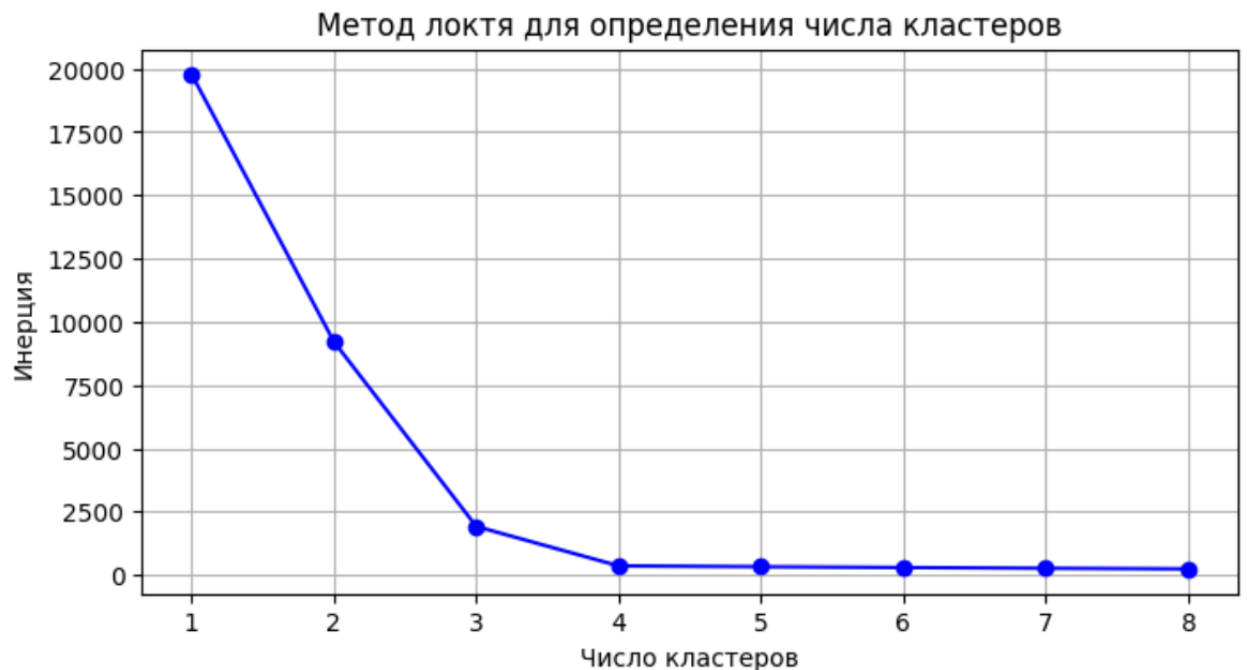
```

plt.plot(range(1, max_clusters + 1), inertias, 'bo-')
plt.xlabel('Число кластеров')
plt.ylabel('Инерция')
plt.title('Метод локтя для определения числа кластеров')
plt.grid(True)
plt.show()

return inertias

# Применение
inertias = find_optimal_clusters(X, max_clusters=8)

```



Задача 2: Сравнение качества кластеризации

```

from sklearn.metrics import silhouette_score, adjusted_rand_score

def evaluate_clustering(X, y_true, methods=['ward', 'complete', 'average', 'single']):
    """
    Оценка качества кластеризации разными методами
    """

    results = []

    for method in methods:
        try:

```

```

clustering = AgglomerativeClustering(n_clusters=3, linkage=method)
labels = clustering.fit_predict(X)
silhouette = silhouette_score(X, labels)
ari = adjusted_rand_score(y_true, labels) if y_true is not None else None
results.append({
    'method': method,
    'silhouette_score': silhouette,
    'adjusted_rand_index': ari
})
except:
    results.append({
        'method': method,
        'silhouette_score': 'error',
        'adjusted_rand_index': 'error'
    })

return pd.DataFrame(results)

# Применение
df_eval = evaluate_clustering(X, y)
print("\nОценка качества кластеризации:")
print(df_eval)

```

Оценка качества кластеризации:

	method	silhouette_score	adjusted_rand_index
0	ward	0.778258	0.712223
1	complete	0.778258	0.712223
2	average	0.778258	0.712223
3	single	0.778258	0.712223

Задача 3: Влияние масштабирования данных

```

from sklearn.preprocessing import StandardScaler

```

```

def test_scaling_effect(X, y):
    """
    Тестирование влияния масштабирования на кластеризацию
    """

    # Исходные данные
    clustering_raw = AgglomerativeClustering(n_clusters=3, linkage='ward')
    labels_raw = clustering_raw.fit_predict(X)
    score_raw = silhouette_score(X, labels_raw)

    # Масштабированные данные
    scaler = StandardScaler()
    X_scaled = scaler.fit_transform(X)
    clustering_scaled = AgglomerativeClustering(n_clusters=3, linkage='ward')
    labels_scaled = clustering_scaled.fit_predict(X_scaled)
    score_scaled = silhouette_score(X_scaled, labels_scaled)

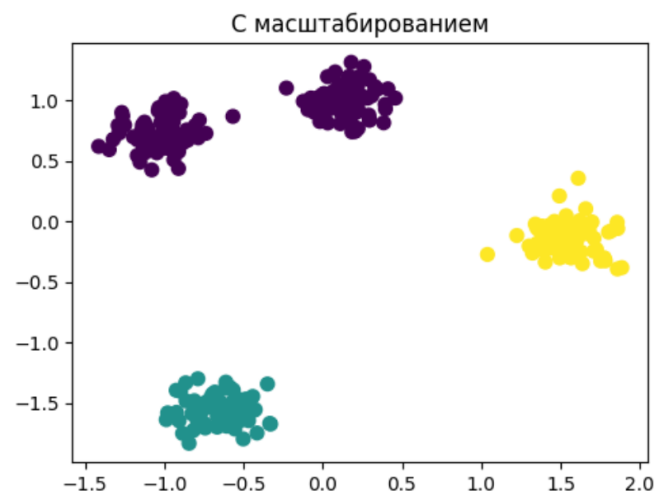
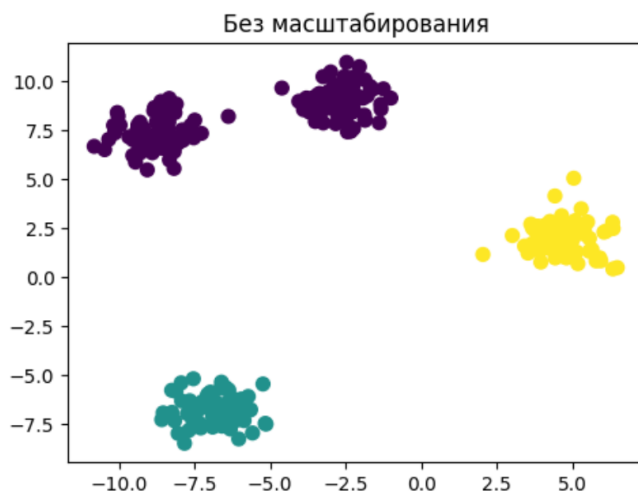
    print(f'Silhouette Score (без масштабирования): {score_raw:.3f}')
    print(f'Silhouette Score (с масштабированием): {score_scaled:.3f}')

    # Визуализация
    fig, axes = plt.subplots(1, 2, figsize=(12, 4))
    axes[0].scatter(X[:, 0], X[:, 1], c=labels_raw, cmap='viridis', s=50)
    axes[0].set_title('Без масштабирования')
    axes[1].scatter(X_scaled[:, 0], X_scaled[:, 1], c=labels_scaled, cmap='viridis',
s=50)
    axes[1].set_title('С масштабированием')
    plt.show()

    # Применение
    test_scaling_effect(X, y)

    Silhouette Score (без масштабирования): 0.778
    Silhouette Score (с масштабированием): 0.764

```



Выводы и рекомендации

1. **Ward linkage** - лучший выбор для сферических кластеров, дает наиболее сбалансированные размеры
2. **Single linkage** - быстрый, работает с нешаровидными данными, но чувствителен к шуму
3. **Average linkage** - хороший компромисс для неевклидовых метрик
4. **Complete linkage** - устойчив к шуму, но может разбивать большие кластеры
5. **Ограничения связности** - ускоряют алгоритм и сохраняют локальную структуру данных
6. **Feature Agglomeration** - эффективен для уменьшения размерности признаков